Advanced Cryptography

Elisabeth Oswald

Verifiable Secret Sharing

Roadmap

SCENARIOS

With Dealer: there is a (trusted) third party called the dealer, which generates a secret (or multiple secrets) as well as the shares, and distributes all shares. (Classical secret sharing)

Without Dealer: parties know their secret(s) and want to compute something jointly based on shares. (Typical MPC)

Implicit Secret: shares are being generated such that they uniquely define a secret without need to construct it. (Threshold cryptography: distributed key generation — we will not go there)

Types of Adversaries

Semi-honest adversary: The corrupted parties co-operate to gather information such as internal states as well as messages that are exchanged during the protocol. The corrupted parties do not deviate from the protocol flow, i.e. they continue to send messages are required by the protocol. The goal is typically to learn the private data of the remaining parties.

Malicious adversary: The corrupted parties co-operate to gather information such as internal states as well as messages. The corrupted parties can deviate from the protocol flow. The goal is typically to subvert the computation of the function.

SCENARIOS FOR CORRUPTION

There may be one or several adversaries and they may or may not collude.

The worst case is if they collude and they (jointly) take over $t - 1$ parties, or, if they take over the dealer.

We model "collusion" by saying that there is a single adversary.

We need to consider what happens if the dealer gets corrupted.

Corrupted dealer

If we consider a semi-honest adversary, then we clearly can no longer hope for any secrecy in relation to the secret or the shares.

If we consider a malicious adversary, then not only have we lost all secrecy, but it may also be that distributed shares are incorrect, so that honest parties cannot reconstruct the secret.

In this case, the best that we can hope for in practice is if the honest parties could identify that the dealer has been corrupted.

$t-1$ Parties corrupted

A (t, n) threshold secret sharing scheme is meant to give security guarantees assuming that at most $t-1$ parties have been corrupted (at least t honest parties are required to reconstruct).

Therefore we are only concerned with the "worst case": we assume $t-1$ corrupted parties.

If we consider a semi-honest adversary, then we want to show that t honest parties can still reconstruct.

If we consider a malicious adversary, then these parties could send wrong messages (including using modified shares). Thus we would like to "detect" if this is the case.

Informal Security Notion

Summarising these considerations, the following three properties are typically hoped for in practice.

1. Reconstruct possible: t uncorrupted users can successfully recover the secret s,

2. Correct secret: if the dealer was honest, then the recovered secret is equal to the secret of the dealer,

3. Share privacy: if the dealer was honest, then $t - 1$ corrupted parties learn no additional information about the secret.

(You can see that the third property is our original security notion.)

Roadmap

Conceptual View

- Secret Sharing: Adversaries

Verifiable Secret Sharing

- The Discrete Logarithm Problem
- Feldman VSS
- Correctness and security

Discrete Logarithm Problem

You know this from El Gamal encryption, and the Digital Signature Algorithm (DSA).

Discrete Logarithm Problem

Let $(G, \times)$ be an Abelian group. Given $g, h \in \mathbb{G}$, find an x (if it exists) such that

$$g^x = h.$$

I used "multiplicative notation" here. Sometimes this is also written as $h = xg$.

The hardness of this problem depends on the group $\mathbb{G}$.

What is "hardness"?

Hard Problems

. . . are problems that nobody want to work on . . . (true but not the meaning in Computer Science).

Hardness: typically relates to the amount of resources (time = calls to an oracle, memory) that are required by an algorithm.

The DL problem is hard because the best known algorithms require resources that grow exponentially in the "size of the DL problem" (the size relates to the chosen prime numbers, assuming g is a generator).

There is also the class of NP-hard problems: they only make good problems to base cryptography on if they can be implemented efficiently on a wide range of devices (typically this is not the case).

(Hard) Discrete Logarithm

$(\mathbb{Z}_n, +)$: given g, h with $h = x \cdot g \pmod{n}$, find x. This is easy: if $gcd(x, n) = 1$ then there exists such an x and we can find it in polynomial time using the extended Euclidean algorithms.

$(\mathbb{Z}_p, \cdot)$: given g, h with $h = g^x \pmod{p}$, find x. This is hard as our best algorithms are (sub) exponential in p (assuming g is a generator).

$(\mathbb{E}_p, +)$: given an elliptic curve $\mathbb{E}$ over $\mathbb{Z}_p$, a base point P, and point Q with $Q = [x]P$, find x. This is harder than the problem over $\mathbb{Z}_p$.

Roadmap

Conceptual View

- Secret Sharing: Adversaries

Verifiable Secret Sharing

- The Discrete Logarithm Problem
- Feldman VSS
- Correctness and security

Feldman VSS — Overview

The scheme "adds a condition" that enables parties to check if the shares that they have been given are "correct". The scheme also sends **broadcast messages** over a public channel.

The condition is an equation over $\mathbb{Z}_p$ that is easy to check for any share, but it does not invertible, so shares are protected (this is where the DL problem comes in).

The condition is also used to reveal a corrupted dealer and corrupted parties.

The `VSS.Share` function proceeds in two steps: share generation/distribution and share checking. The `VSS.Reconstruct` function also proceeds in two steps: share checking and reconstruction.

Share distribution:

To share a secret message m the dealer chooses uniform coefficients $a_0, a_1, \ldots, a_{t-1} \in \mathbb{Z}_q$ giving rise to the secret polynomial $f(x) = \sum_{j=0}^{t-1} a_j x^j$.

Share i is the i-th point on the polynomial: $s_i = f(i) = \sum_j a_j i^j$. Party P_i gets their respective shares via a private channel between the dealer and the party.

Furthermore the dealer broadcasts the values $A_0 = g^{a_0}, \ldots, A_{t-1} = g^{a_{t-1}}$, and the value $c = H(a_0) \oplus m$.

$H()$ must behave like a random function (i.e. we choose a hash function with suitable output size).

Share verification: via

$$g^{s_i} \equiv \prod_{j=0}^{t-1} (A_j)^{i^j} \pmod{q}.$$

1. If party P_i finds that their share does not satisfy the equation then they broadcast a complaint.
   - Abort: If more than $t-1$ parties complain, then honest parties abort the protocol.
2. All parties P_i who complained receive their s_i again via a public broadcast.
   - Abort: If the dealer refuses to respond.
   - Abort: If any broadcast value s_i does not satisfy the verification equation.

Reconstruction will only be carried out if the dealer has not been disqualified previously (i.e. the share protocol successfully completed).

Share verification:

We need at least t honest parties (i.e. parties who did not complain).

The share of each user gets verified according to the share verification equation: We must verify shares because it could be that a corrupted party did not complain during the Share protocol.

If at least t shares can be verified, then they proceed to reconstruct, otherwise they abort.

Reconstruct:

Reconstruct works according to `Stn.Reconstruct` to derive a_0.

Using a_0 c the t honest users can reveal via $m = H(a_0) \oplus c$.

ROADMAP

Conceptual View
- Secret Sharing: Adversaries

Verifiable Secret Sharing
- The Discrete Logarithm Problem
- Feldman VSS
- Correctness and security

CORRECTNESS

An honest dealer is never disqualified (we explain this in the security argument).

Assuming an honest dealer: the value that can be recovered by t honest parties who hold verified shares is equal to m. This is because we are basing the scheme on Shamir's scheme, and we proved before that there is only one polynomial and therefore a_0 is uniquely defined.

With a_0 and the public knowledge of c and $H()$ they can find m.

Correctness corresponds to the first and the second of the properties that we want from a VSS.

SECURITY ARGUMENT, CONT.

If the dealer is honest then:

$$\prod_{j=0}^{t-1} (A_j)^{i^j} = \prod_{j=0}^{t-1} (g^{a_j})^{i^j} = g^{\sum_{j=0}^{t-1} a_j \cdot i^j} = g^{f(i)} = g^{s_i} \pmod{q}.$$

Therefore all parties can in fact compute g^{s_i} for all values of i.

But only party P_i can check that their s_i satisfies the verification equation.

The DL problem keep s_i safe from all parties P_j ($j \neq i$).

SECURITY ARGUMENT, CONT.

A honest dealer is never disqualified. (If no more than $t - 1$ parties are corrupted.) Honest parties will not complain when they get a correct share. An honest dealer gives out correct shares.

The protocol tolerates at most $t - 1$ corrupted parties. Therefore no more than $t - 1$ complaints can emerge, which means that the protocol `VSS.Share` will complete.

If more than $t - 1$ complaints emerge: either there are more than $t - 1$ corrupt parties (in which case our protocol does not provide any security guarantees), or, the dealer was corrupt. The protocol therefore aborts.

SECURITY ARGUMENT, CONT.

If the dealer does not respond to a complaint, then we also assume they are corrupted, and we abort.

"Obviously" if a dealer is corrupted, but follows the protocol, then it is impossible to identify this.

We were only able to show that an honest dealer though cannot be disqualified by $t - 1$ corrupted parties (which is what our protocol is meant to tolerate).

What remains to be shown is that $t - 1$ corrupt users do not learn anything about the secret (or the shares of the other parties).

SECURITY ARGUMENT, CONT.

We know that $t-1$ corrupted parties cannot reconstruct the secret polynomial (we showed this already for Shamir's reconstruct).

However g^{a_0} is known to all parties. Any of the corrupted parties can therefore try and solve the discrete-logarithm problem over $\mathbb{Z}_q$.

We can only give computational security.

Assuming that the discrete-logarithm problem is hard, even $t-1$ corrupt parties cannot extract a_0 from g^{a_0}. Therefore they only have g^{a_0}, and they have c.

But c is close to a one-time pad encryption of m because $H(a_0)$ looks like a random value. Therefore the corrupted parties do not learn any additional information about m by seeing c.